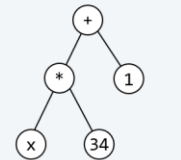


4 Datenorientierter Systementwurf

'x'	→	ident
'*'	→	mult
'3'	}	number
'4'		
'+'	→	plus
'1'	→	number



Annahme: $x = 2$

- 4.1 Denkmodell
- 4.2 Formale Sprachen und Grammatiken
- 4.3 Systementwurf mit ATGen
- 4.4 Transformation ATGen in Java

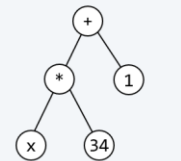
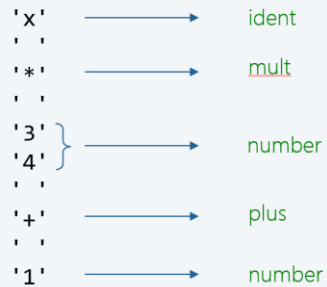
4 Datenorientierter Systementwurf

4.1 Denkmodell

4.2 Formale Sprachen und Grammatiken

4.3 Systementwurf mit ATGen

4.4 Transformation ATGen in Java



Annahme: $x = 2$

Zum Denkmodell des datenorientierten Systementwurfs

- in der Praxis findet man häufig Aufgabenstellungen, in denen es darum geht, dass **ein** Strom **strukturierter Datenobjekte** zu verarbeiten ist
- Softwaresysteme für solche Aufgabenstellungen können auf elegante und systematische Weise mit Hilfe von **attribuierten Grammatiken** **spezifiziert** werden
- darauf aufbauend kann dann die **Systemarchitektur** und die **Systemimplementierung** abgeleitet werden
- die solcherart entworfenen Lösungen beschreiben die Transformation eines gegebenen Stroms von Datenobjekten in eine andere Form
- wir sprechen deshalb von **transformationsorientierter Software**
- solche Systementwürfe können **automatisch** in Algorithmensysteme transformiert werden

Beispiele für Strom strukturierter Datenobjekte

Pascal-Programm

```
PROGRAM P;  
  VAR a, b: INTEGER;  
BEGIN  
  Read(a); Read(b);  
  IF a < b THEN ...  
END.
```

Mail-Strom

```
marcel hirscher hat seine alpine  
karriere offiziell beendet zzzz  
ausbruch des corona... zzzz auftakt  
der... zzzz zzzz
```

URL

```
http://oe.orf.at/stories  
https://www.fh-oe.at  
ftp://lorem.ipsum.com:3343
```

Bewegungsdatei

```
(100,100,100)  
10 L (500,100,100)  
20 C (700,300,100)(500,500,100)  
10 L (100,500,100)  
E
```

Arithmetische Ausdrücke

```
x * 3421 + 1  
3 * (4 + 5)  
(hs + d*q - dm*si) mod q
```

E-Mail-Adressen

```
alice@yahoo.org  
bob@whitehouse.gov  
gmail@chucknorris.com
```

Zum Denkmodell des datenorientierten Systementwurfs

- das Denkmodell für den datenorientierten Entwurf in der Softwareentwicklung geht davon aus, dass aufbauend auf der **Struktur (Syntax)** eines Eingabedatenstroms seine **Bedeutung (Semantik)** und der angestrebte **Transformationsprozess** definiert und daraus die Systemarchitektur abgeleitet werden kann
- zur Beschreibung der **syntaktischen Struktur** des Eingabedatenstroms können wir das, aus dem Gebiet der formalen Sprachen bekannte Konzept der **Grammatiken** und für die Beschreibung des **Transformationsprozesses** das der **attributierten Grammatiken** heranziehen
- der **Syntaxanalyse** des Eingabedatenstroms kommt eine zentrale Bedeutung im Rahmen des Transformationsprozesses zu

Pascal-Programm

```
PROGRAM P;  
  VAR a, b: INTEGER;  
BEGIN  
  Read(a); Read(b);  
  IF a < b THEN ...  
END.
```

```
Program = ident [ UsesClause ] Block ".".  
Block = DeclPart BlockStat.  
Stat = AssignStat | BlockStat | IfStat | ... .  
AssignStat = Lvalue ":" Expr.  
IfStat = "IF" Expr "THEN" Stat [ "ELSE" Stat ].  
BlockStat = "BEGIN" Stat { ";" Stat } "END".  
...
```

Mail-Strom

```
marcel hirscher hat seine alpine  
karriere offiziell beendet zzzz  
ausbruch des corona... zzzz auftakt  
der... zzzz zzzz
```

```
S = { M } L.  
M = W { W } "zzzz".  
W = C { C } " " { " " } .  
L = "zzzz".  
C = "a" | "b" | ... | "z".
```

URL

```
http://oe.orf.at/stories  
https://www.fh-oe.at  
ftp://lorem.ipsum.com:3343
```

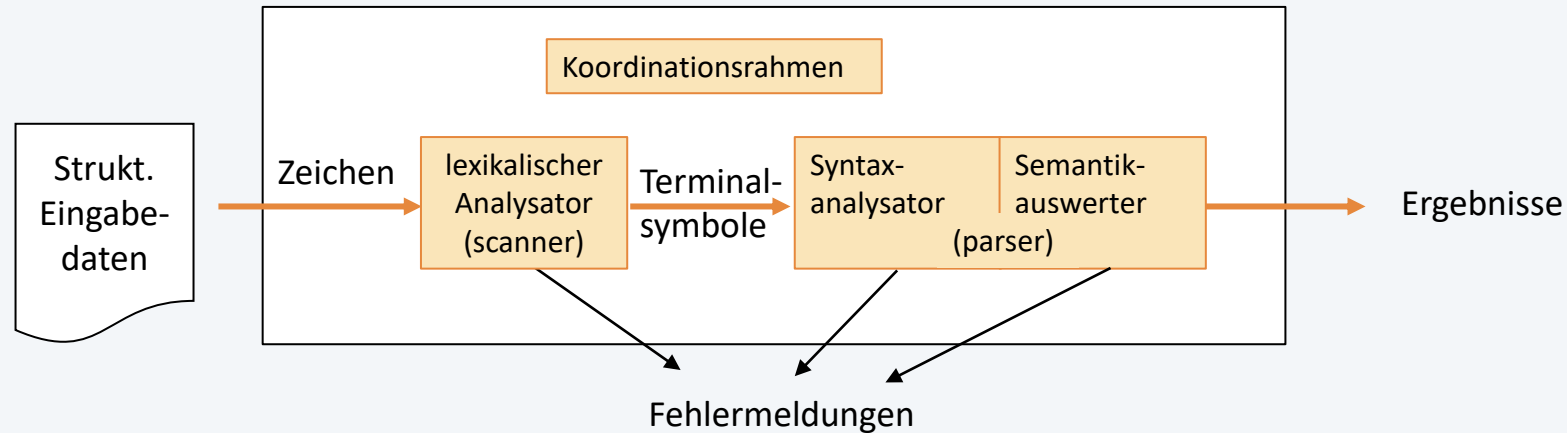
```
Url = Protocol "://" Host [ ":" Port ] [ "/" Path ]  
Protocol = "http" | "https" | "ftp".  
Host = ident { "." ident } .  
Port = number.  
Path = ident.
```

Zum Denkmodell des datenorientierten Systementwurfs

- die Konstruktion datenorientierter **Systemarchitekturen** erfordert die Entwicklung von **Analysatoren**, mit deren Hilfe die syntaktische Korrektheit der zu verarbeitenden Eingabedatenströme geprüft werden kann (den so genannten **lexikalischen Analysator (engl. *scanner*)** und den so genannten **Syntaxanalysator (engl. *parser*)**)
- um einen Eingabedatenstrom verarbeiten zu können, d. h. die gewünschten Ergebnisse aus ihm ermitteln bzw. diesen in eine andere Gestalt transformieren zu können, ist es notwendig, auch die semantischen Aspekte des Eingabedatenstroms zu berücksichtigen (deshalb benötigen wir den so genannten **Semantikauswerter**)

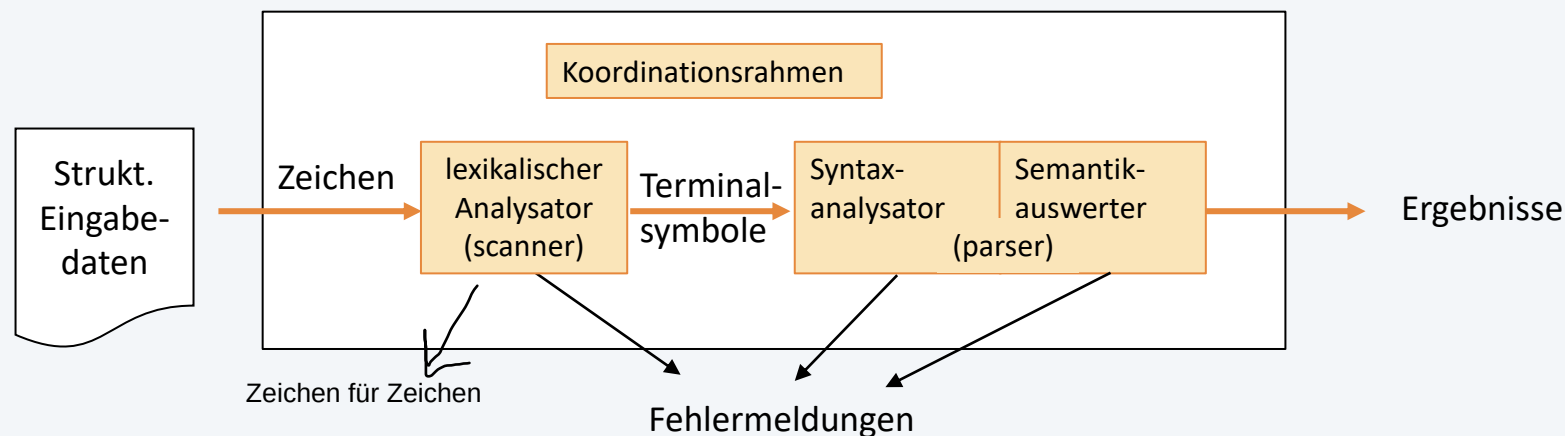
Zum Denkmodell des datenorientierten Systementwurfs

Grobarchitektur



- Grundstruktur der Systemarchitektur bleibt immer gleich (Referenzarchitektur)
- Architekturmuster Pipes & Filters

Zum Denkmodell des datenorientierten Systementwurfs



Beispiel

Eingabedaten

"x * 34 + 1"

lexik. Analysator:
Zeichen für Zeichen einlesen
-- Kommentare und Leerzeichen
fallen bereits hier weg

Zeichen

'x'

' '

'*'

' '

'3'

'4'

' '

'+'

' '

'1'

Terminalsymbole

ident

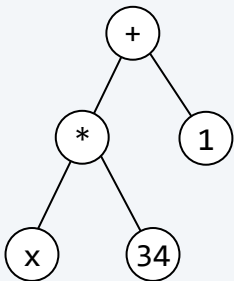
mult

number

plus

number

Operatorbaum



Annahme: x = 2

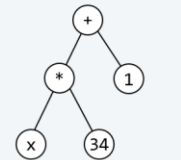
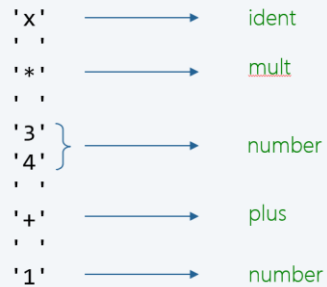
für die Syntaxprüfung wird nur geprüft, ob Terminalsymbole
in der richtigen Reihenfolge sind

Ergebnis

69

+ Werte, wenn sie relevant sind
z.B. bei number muss man die number speichern, bei plus nicht

4 Datenorientierter Systementwurf



Annahme: $x = 2$

4.1 Denkmodell

4.2 Formale Sprachen und Grammatiken

4.3 Systementwurf mit ATGen

4.4 Transformation ATGen in Java

- Grundelement ist ein Zeichen oder **Symbol** (**Terminalsymbol**)
- das **Alphabet** ist eine endliche Menge von Zeichen oder Symbolen

$$V_1 = \{"0", "1"\} \quad V_2 = \{":", ";", "-", "(", ")\}"$$

$$V_3 = \{"if", "else", ":", "<", \text{ident}, \dots\}$$

- durch Hintereinanderschreiben entstehen Zeichen- bzw. **Symbolketten**

für V_1 z.B. 010 0110 01010 0000111101010101

für V_2 z.B. :-) :- (:-----)))))

für V_3 z.B. if a < b then min := a else min := b

- einen **Eingabedatenstrom** fassen wir als Zeichen- bzw. **Symbolkette** auf
- die Menge V^+ über einem Alphabet V ist Menge aller nichtleeren Ketten, die sich aus Elementen von V bilden lassen
- die Menge V^* über einem Alphabet V ist Menge aller Ketten einschließlich der leeren Kette ε ($V^* = V^+ \cup \{\varepsilon\}$)

Definition **Formale Sprache**

- eine formale Sprache **L** über einem Alphabet **V** ist eine Teilmenge von **V***:

$$L \subseteq V^*$$

V^* ist die Menge aller Aneinanderreihung von Zeichen

L ist die Menge aller Sätze.

Ein Satz ist ein sinnvoller Teil dieser Sprache

Es können alle gültigen Sätze explizit angegeben werden.

Allerdings bei komplexen Sprachen geht sich das nicht aus
man braucht Grammatik um eine Sprache zu definieren

- ihre Elemente heißen **Sätze**
- Beispiel: Sprache mit vier Sätzen

$$V = \{":", ";", "-", "(", ")"\}$$

$$V^* = \{\epsilon, ":", ";", "-", ")", "(", ":", ";", ":", ":", "... \}$$

$$L = \{":-)", ":-(", ";:-)", ";:-(" \}$$

- Teilmenge kann auf verschiedene Arten definiert werden
 - Aufzählung (für sehr einfache Sprachen mit wenigen Sätzen, siehe Beispiel)
 - Grammatik
 - Automaten

Eine **Grammatik** G mit dem **Satzsymbol** S , geschrieben $G(S)$, ist eine endliche, nicht leere Menge von **Ersetzungsregeln**, die zwei Arten von Symbolen enthalten:

- **Terminalsymbole**, die in Sätzen vorkommen,
- und **Nonterminalsymbole**, die der Strukturbeschreibung dienen.

Das Satzsymbol S ist ein ausgezeichnetes Nonterminalsymbol, das auf mindestens einer linken Seite der Ersetzungsregeln vorkommt.

Beispiel: Sprache mit vier Sätzen

	$:-)$	$;-)$	$:-()$	$;-()$
X	$:-)$	$;-)$	$:-()$	$;-()$
Y	$:-)$	$;-)$	$:-()$	$;-()$
Z	$:-)$	$;-)$	$:-()$	$;-()$

Grammatik $G(S)$:

```
S = X Y Z.  
X = ":".  
X = ";".  
Y = "-".  
Z = ")".  
Z = "(".
```

```
 $V_T = \{":", ";", "-", ")", "("\}$   
 $V_N = \{S, X, Y, Z\}$   
Satzsymbol  $S$ 
```

- eine Ersetzungsregel besteht aus einem Nonterminalsymbol A und einer Symbolkette α , einer Folge aus Terminal- und/oder Nonterminalsymbolen
- eine Regel wird geschrieben als $A = \alpha$. und gelesen als „ A ist definiert als α “ oder „ A kann ersetzt werden durch α “.

$$\underbrace{S}_{A} = \underbrace{X Y Z}_{\alpha}$$

$$\underbrace{X}_{A} = \underbrace{":."}_{\alpha}$$

Dabei heißt A „linke Seite“ und α „rechte Seite“ der Regel.

- gibt es mehrere Möglichkeiten für die Ersetzung von A , kann anstelle von z. B. zwei Regeln $A = \alpha$. und $A = \beta$. auch $A = \alpha \mid \beta$. geschrieben werden; das wird gelesen als „ α oder β “, dabei werden α und β als Alternativen von A bezeichnet

$\begin{array}{l} S = X Y Z. \\ X = ":." \\ X = " ; ". \\ Y = "-". \\ Z = ")". \\ Z = "(". \end{array}$	$\left. \begin{array}{l} \\ \\ \end{array} \right\} \begin{array}{l} X = ":." \mid " ; ". \\ \\ \\ Z = ")". \mid "(". \end{array}$	$\Rightarrow$	$\begin{array}{l} S = X Y Z. \\ X = ":." \mid " ; ". \\ Y = "-". \\ Z = ")". \mid "(". \end{array}$	$\begin{array}{l} S = X "-." Z. \\ X = ":." \mid " ; ". \\ Z = ")". \mid "(". \end{array}$	$S = (":." \mid " ; ") "-." (")". \mid "(").$
--	---	---------------	--	---	--



Beispiel: Musiknoten (Version 1)

- drei Noten
- Notennamen: c, d, e, f, g, a, h
- Notenwerte: 1, 2, 4, 8

g1, g2, a2

c1, d2, e4

a2, a4, a8

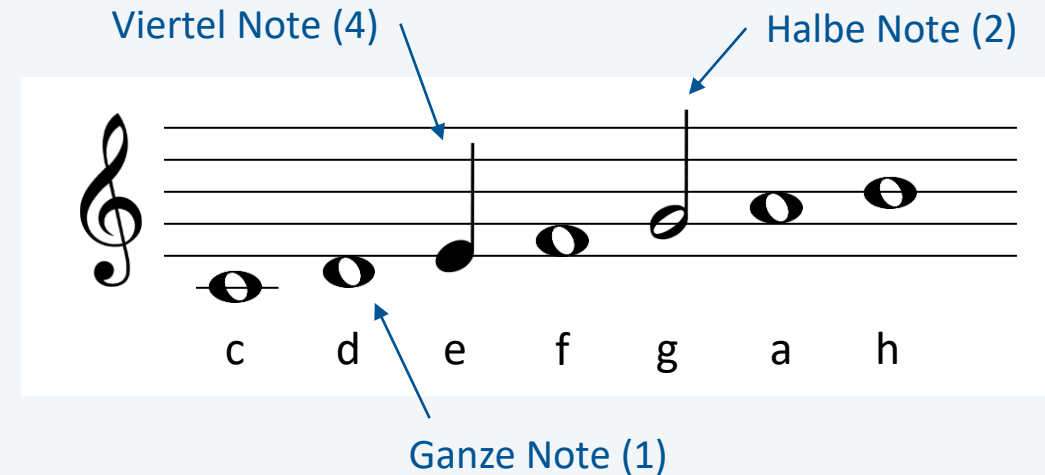
- Grammatik G(Bar)

$V_T = \{",", "c", "d", "e", "f", "g", "a", "h", "1", "2", "4", "8"\}$

$V_N = \{S, N, D\}$

$S = ND \text{ '}' ND \text{ '}' ND$
 $N = 'c' \mid 'd' \mid 'e' \mid 'f' \mid 'g' \mid 'a' \mid 'h'$
 $D = '1' \mid '2' \mid '4' \mid '8'$

S = Satzsymbol
N = Note
D = Dauer



Menge der Terminalsymbole

Menge der Nonterminalsymbole
und Satzsymbol

Ersetzungsregeln

- **Nonterminalsymbole** sind all jene Symbole, die auf den linken Seiten der Ersetzungsregeln einer Grammatik stehen und als Abstraktionsmittel zur Gliederung der Sätze verwendet werden (das wichtigste Nonterminalsymbol ist das Satzsymbol)
- **Terminalsymbole** sind jene Symbole, die nur auf rechten Seiten von Grammatikregeln und somit als Elemente in den Sätzen der Sprache der Grammatik vorkommen. Die Menge der Terminalsymbole bildet das (Terminalsymbol-)Alphabet der Sprache
- Aus dem Satzsymbol S kann man durch **Ableitungen** (fortgesetzte Anwendung der Ersetzungsregeln) Symbolketten, so genannte Satzformen bilden. Eine Satzform, die nur mehr Terminalsymbole enthält, ist ein Satz
- Die **Sprache** $L(G)$ einer Grammatik $G(S)$ repräsentiert die Menge aller Sätze, die sich mit Hilfe ihrer Ersetzungsregeln aus dem Satzsymbol S erzeugen oder ableiten lassen

Aus dem Satzsymbol S kann man durch Ableitungen Symbolketten, so genannte Satzformen bilden

Grammatik $G(S)$:

$S = X Y Z.$
 $X = " : " \mid " ; " .$
 $Y = " - " .$
 $Z = ") " \mid " (" .$

Ableitung

	$X = " : "$	$Y = " - "$	$Z = ") "$												
$S \Rightarrow$	X	Y	Z	$\Rightarrow$	$:$	Y	Z	$\Rightarrow$	$:$	$-$	Z	$\Rightarrow$	$:$	$-$	$)$
$S \Rightarrow$	X	Y	Z	$\Rightarrow$	$:$	Y	Z	$\Rightarrow$	$:$	$-$	Z	$\Rightarrow$	$:$	$-$	$($
$S \Rightarrow$	X	Y	Z	$\Rightarrow$	$;$	Y	Z	$\Rightarrow$	$;$	$-$	Z	$\Rightarrow$	$;$	$-$	$)$
$S \Rightarrow$	X	Y	Z	$\Rightarrow$	$;$	Y	Z	$\Rightarrow$	$;$	$-$	Z	$\Rightarrow$	$;$	$-$	$($

Sätze

Die Sprache $L(G)$ ist Menge aller Sätze

$L(G) = \{ " : -) " , " : - (" , " ; -) " , " ; - (" \}$

Grammatik G(S)

$S = A ";"$.
 $A = "a" B \mid B B "b"$.
 $B = "b" \mid "a" "b"$.

$V_T = \{"a", "b", ";"\}$
 $V_{NT} = \{S, A, B\}$

Ableitungen

$S \Rightarrow A ; \Rightarrow a B ; \Rightarrow a b ;$

$S \Rightarrow A ; \Rightarrow a B ; \Rightarrow a a b ;$

$S \Rightarrow A ; \Rightarrow B B b ; \Rightarrow b B b ; \Rightarrow b b b ;$

$S \Rightarrow A ; \Rightarrow B B b ; \Rightarrow b B b ; \Rightarrow b a b b ;$

$S \Rightarrow A ; \Rightarrow B B b ; \Rightarrow a b B b ; \Rightarrow a b b b ;$

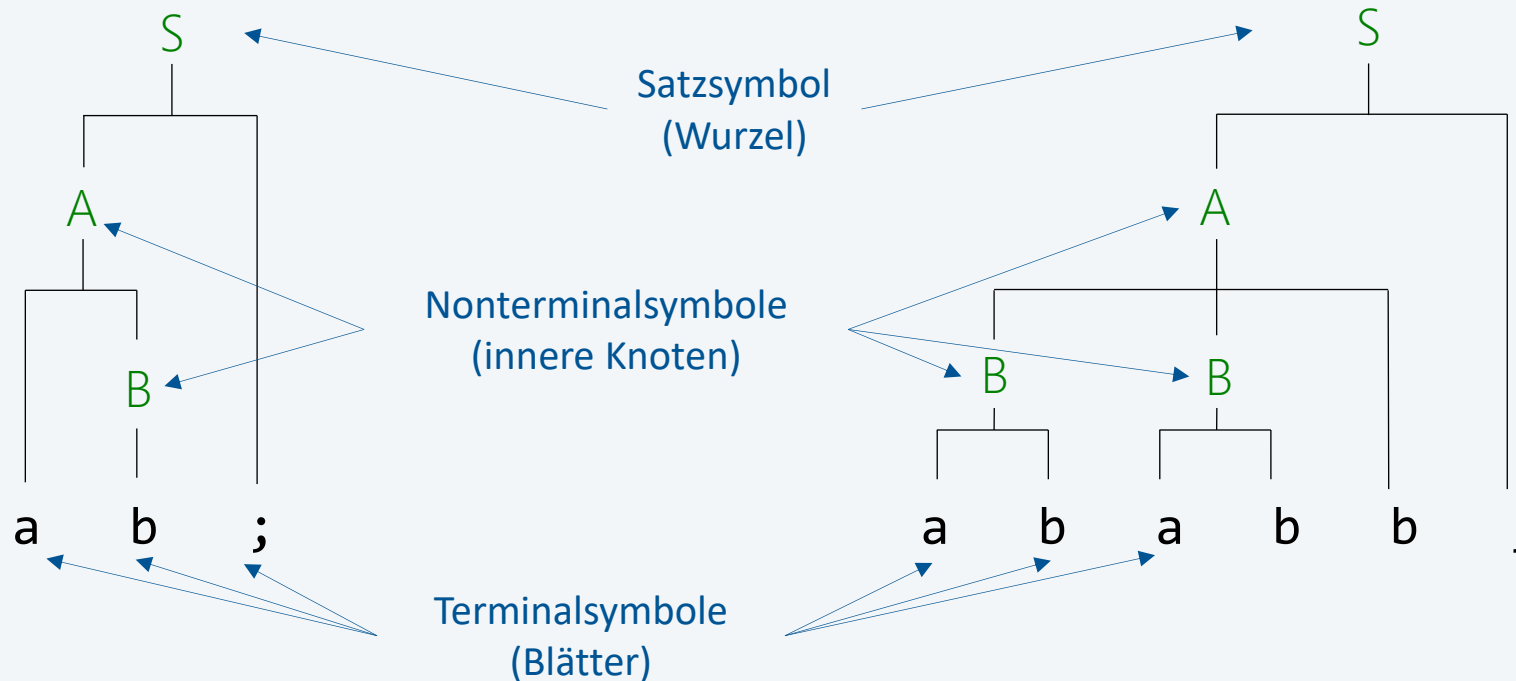
$S \Rightarrow A ; \Rightarrow B B b ; \Rightarrow a b B b ; \Rightarrow a b a b b ;$

Syntaxbaum (Parse-Baum)

Zeigt die Ableitungsstruktur für einen Satz einer Grammatik

Beispiele für Grammatik $G(S)$

$S = A \text{ ";"}$.
 $A = \text{"a" } B \mid B B \text{ "b"}$.
 $B = \text{"b" } \mid \text{"a" "b"}$.



Grammatiken können auch rekursive Ersetzungsregeln haben

Man unterscheidet direkte und indirekte Rekursion

Direkte Rekursion (mit $\alpha, \beta, \gamma \in V$):

- linksrekursive Regeln $A = \alpha \mid A \beta$.
- rechtsrekursive Regeln $A = \alpha \mid \beta A$.
- zentralrekursive Regeln $A = \alpha \mid \beta A \gamma$.

Beispiel: Grammatik $G(S)$ für Binärzahlen und leere Kette

$S = B$.
 $B = "0" B \mid "1" B \mid \varepsilon$.

$V_T = \{"0", "1"\}$
 $V_{NT} = \{S, B\}$

Epsilon steht für ein leeres Symbol bzw. kein Symbol
wird gebraucht für den nicht-rekursiven Zweig der Rekursion

Ableitung (Beispiele)

$S \Rightarrow 0 B \Rightarrow 0 \varepsilon = \mathbf{0}$

$S \Rightarrow 1 B \Rightarrow 1 \varepsilon = \mathbf{1}$

$S \Rightarrow 1 B \Rightarrow 1 0 B \Rightarrow 1 0 \varepsilon = \mathbf{1 0}$

$S \Rightarrow 1 B \Rightarrow 1 0 B \Rightarrow 1 0 1 B \Rightarrow 1 0 1 \varepsilon = \mathbf{1 0 1}$

...



Menge aller Sätze, die mit "a" beginnen, mit "c" oder "d" enden und bei denen zwischen dem ersten und letzten Zeichen beliebig viele "b" vorkommen dürfen.

Grammatik:

Terminalsybole = {'a', 'c', 'd'}

$S = 'a' B E$ // E für Ende

$B = 'b' B \mid \text{epsilon}$

$E = 'c' \mid 'd'$

auch möglich:

$S = 'a' B$

$B = 'b' B \mid T$

$T = 'c' \mid 'd'$

Beispiele:

abc

abbbbd

abbbbbbbbc

ac // auch 0 sind beliebig viele

Beispiel: Grammatik für Palindrom

Ein Palindrom ist eine Zeichenkette, die vorwärts und rückwärts gelesen identisch ist, z.B. otto oder madamimadam

Der Einfachheit halber betrachten wir Palindrome, die sich mit dem Alphabet $V = \{ "0", "1" \}$ beschreiben lassen, z.B. 0, 0110, 11011

Grammatik:

$P = '0' \mid '1' \mid '0' P '0' \mid '1' P '1' \mid \text{epsilon}$

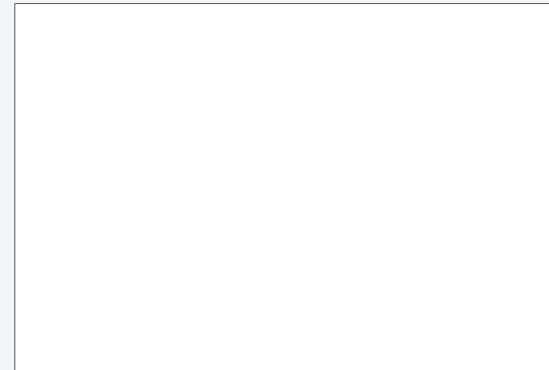
-- das epsilon ist nötig, damit

0 1 1 0

funktioniert

in der Mitte muss eine leere Zeichenkette
als Palindrom gesehen werden.

Beispiele:

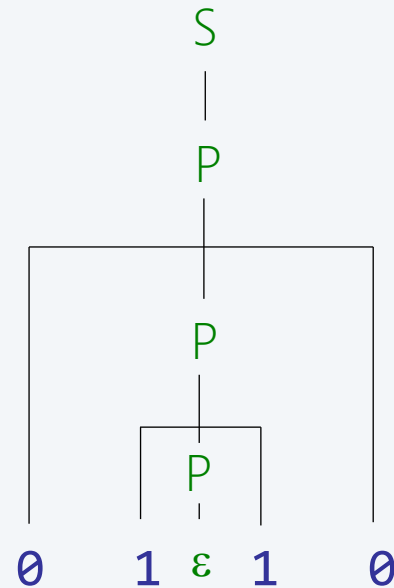
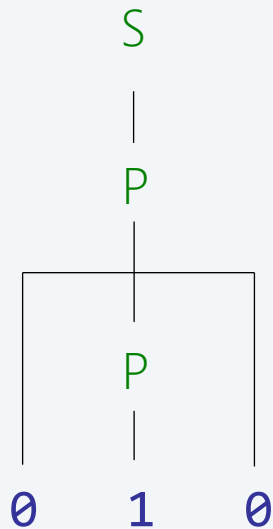


Beispiel: Grammatik für Palindrom

Syntaxbäume

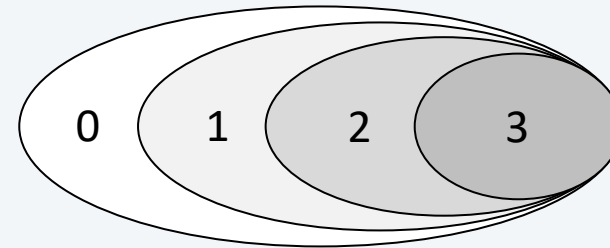
$S = P.$

$P = \varepsilon \mid "0" \mid "1" \mid "0" P "0" \mid "1" P "1".$



Der Linguistiker **Noam Chomsky** hat eine Sprachtypologie, die nach ihm benannte Chomsky-Hierarchie der Sprachen, definiert und die Grammatiken nach der Form ihrer Ableitungsregeln in vier **Typen** eingeteilt, die er von 0 bis 3 durchnummeriert hat:

- die unbeschränkten (Typ 0)
- die kontextsensitiven (Typ 1)
- die **kontextfreien** (Typ 2)
- die **regulären** (Typ 3)



Nur die beiden einfachsten Sprach- bzw. Grammatikklassen – jene vom Typ 3, die **regulären**, und jene vom Typ 2, die **kontextfreien** – sind für die datenorientierte Programmierung relevant

4.2.1 Reguläre Grammatiken

Eine Grammatik heißt regulär (Typ 3), wenn sie sich durch Ersetzungsregeln der folgenden Art ausdrücken lässt:

$A = b B.$

$A = a.$

$V_T = \{a, b\}$

$V_{NT} = \{A, B\}$

- für das Satzsymbol ist zusätzlich $S = \varepsilon$. erlaubt

Beispiel: Grammatik der Bezeichner

- ein Bezeichner besteht aus Buchstaben **letter** und Ziffern **digit**, von denen das erste Zeichen ein Buchstabe sein muss.
- reguläre Grammatik $G(S)$

$S = \text{letter} \mid \text{letter } R.$

$R = \text{letter} \mid \text{digit} \mid \text{letter } R \mid \text{digit } R.$

$V_T = \{\text{letter}, \text{digit}\}$

$V_{NT} = \{S, R\}$

- reguläre Grammatiken können als **regulärer Ausdruck** definiert werden

$\text{letter} (\text{letter} \mid \text{digit})^*$

- Äquivalent zu regulären Grammatiken
- Beschreibung einer Menge von Zeichenketten (formalen Sprache)

Definition

- die leere Kette (ε) ist ein regulärer Ausdruck
- ein einzelnes Zeichen oder *wildcard* $.$ ist ein regulärer Ausdruck
- wenn α und β reguläre Ausdrücke sind, dann sind auch folgende Ausdrücke regulär:

$\alpha \beta$ die Konkatenation von α und β

$\alpha \mid \beta$ eine der beiden Alternativen von α und β (auch $\alpha + \beta$)

α^* die null- oder mehrmalige Wiederholung von α (*closure*)

(α) Zusammenfassung von Teilausdrücken (z.B. $(\alpha \mid \beta)^*$)

Beispiel: $a b^* (c \mid d)$

- beschreibt die Menge $\{ac, ad, abc, abd, abbc, abbd, \dots\}$

Beispiele

Regulärer Ausdruck	Sätze der Sprache	keine Sätze
$aa \mid baab$	aa baab	jede andere Kette
$.u.u.u.$	cumulus	succubus
$. * 0 \dots$	1000234 98701234	111111111 403982772
$ab^*(c \mid d)$	ad abc abbbbbc	abcd
$a(a \mid b)aab$	aaaab abaab	jede andere Kette
$(ab)^*a$	a abababa	aa abbba

Zusätzliche Operatoren ermöglichen kürzere Ausdrücke

Operation	Beispiel	Sätze der Sprache	Keine Sätze
Zeichenmengen	<code>[abc][xyz]</code>	az bx	xa
	<code>[A-Za-z][a-z]*</code>	Hagenberg schnell	maxN
Negation	<code>[^aeiou]*</code>	Typ	Type
Meta-Zeichen	<code>\[.*\]</code>	[i]	[j[
Einmal/mehrmal	<code>a(bc)+</code>	abc abcbc	a
Null-/einmal	<code>a(bc)?</code>	a abc	abcbc
Genau n -mal	<code>[0-9]{5}</code>	31415	1234 123456

Anwendungen regulärer Grammatiken / regulärer Ausdrücke

- Pattern Matching
- Betriebssysteme (Unix, Windows) und Bibliotheken
- Lexikalische Analyse



Beispiel: E-Mail-Adresse

Eine E-Mail-Adresse ist folgendermaßen aufgebaut

1. Sequenz von Buchstaben
2. Zeichen "@"
3. Sequenz von Buchstaben, gefolgt von Zeichen "." (beliebige Anz. von Wdh.)
4. "at" oder "com" (vereinfacht)

Welche der folgenden Zeichenketten sind E-Mail-Adressen?

alice@yahoo.com	✓
hagenberg	X
mail@chuck.norris.com	✓
bob@whitehouse	X
john.doe@fhooe.at	X

Regulärer Ausdruck (Alternativen werden oft mit | statt mit + geschrieben)

[a-z]+@([a-z]+\.)+(at|com)

Anwendung 1: Pattern Matching



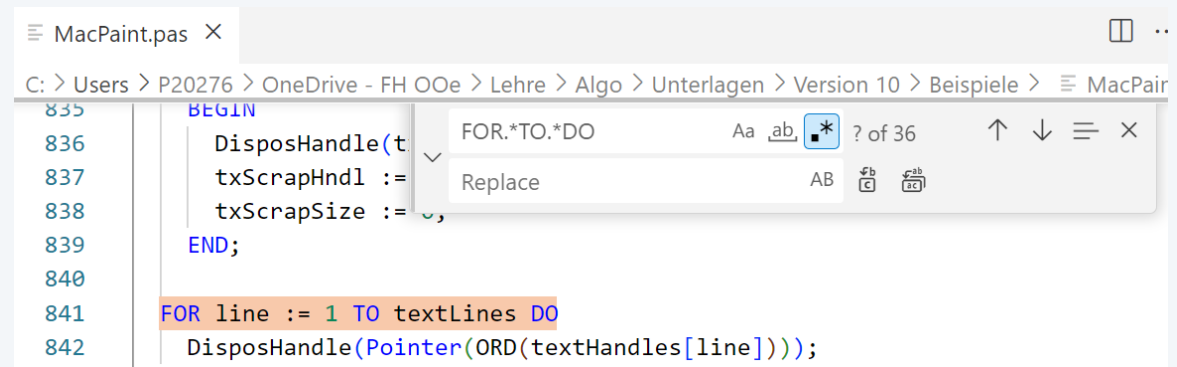
Beispiel: Datum in Form YYYY-MM-DD (z.B. 2020-04-01)

Beispiel: Telefonnummern (z.B. (800) 555-1212)

- Standardlösungen für die Mustersuche in Zeichenketten (z.B. Dateien)
- Unix: grep (Generalized Regular Expression Pattern matcher)
 - entwickelt von Ken Thompson (Entwickler von Unix, Turing Award)
- Windows: findstr

```
Regular expression quick reference:  
.  
*      Wildcard: any character  
*      Repeat: zero or more occurrences of previous character or class  
^      Line position: beginning of line  
$      Line position: end of line  
[class] Character class: any one character in set  
[^class] Inverse class: any one character not in set  
[x-y]   Range: any characters within the specified range  
\\x     Escape: literal use of metacharacter x  
\\<xyz  Word position: beginning of word  
xyz\\>  Word position: end of word
```

- Suche in VSCode



Klasse String

```
boolean matches(String regex)
```

Beispiel: E-Mail-Adresse

```
String[] a = {"alice@yahoo.com", "hagenberg", ...};
```

```
final String pattern = "[a-z]+@([a-z]+\\.\\.)+(at|com)";  
for (String s : a) {  
    System.out.format("%-25s", s);  
    if (s.matches(pattern)) {  
        System.out.println("✓");  
    } else {  
        System.out.println("X");  
    }  
}
```

erster escape ist vom Java-String
zweiter escape ist vom regulären Ausdruck

alice@yahoo.com	✓
hagenberg	X
mail@chuck.norris.com	✓
bob@whitehouse	X
john.doe@fhoee.at	X

Beispiel: Datum in Form YYYY-MM-DD (z.B. 2020-04-01)

```
final String pattern = "[0-9]{4}-[0-9]{2}-[0-9]{2}";

IO.print("Enter a date (yyyy-mm-dd): ");
String dateStr = IO.readLine();
while (!dateStr.matches(pattern)) {
    IO.print("Invalid date format. Enter a date (yyyy-mm-dd): ");
    dateStr = IO.readLine();
}

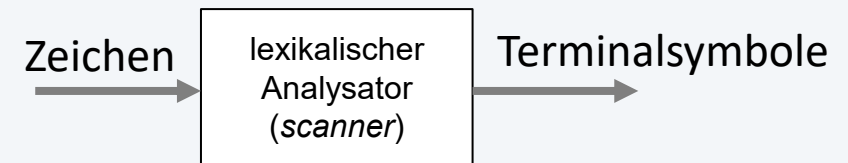
LocalDate date = LocalDate.parse(dateStr);
IO.println("Day of month: " + date.getDayOfMonth());
IO.println("Day of week: " + date.getDayOfWeek());
IO.println("Day of year: " + date.getDayOfYear());
```

```
Enter a date (yyyy-mm-dd): 2025.04.01
Invalid date format. Enter a date (yyyy-mm-dd): 2025-04-01
Day of month: 1
Day of week: TUESDAY
Day of year: 91
```

Anwendung 3: Lexikalische Analyse

Aufgaben der lexikalischen Analyse

- Liefert Terminalsymbole
- Überliest bedeutungslose Zeichen



Beispiel: Pascal-Programm

```
IF x <= min THEN BEGIN
  x := 42
END; (* IF *)
```

Terminalsymbole haben eine Struktur

```
then = "t" "h" "e" "n".
number = digit | digit rest.
rest = digit | digit rest.
```

Erkennung ist nicht Teil der Syntaxanalyse

- einfachere Syntaxanalyse ...*"i"* (*"f"* Expr ... | letter ...
- Überlesen von Leerzeichen ...*"if"* { Blank } Expr { Blank } ...
- für Terminalsymbole reichen reguläre Grammatiken

Symbolfolge

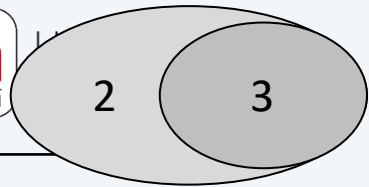
Symbol	Wert
if	
ident	"x"
lessEq	
ident	"min"
then	
begin	
ident	"x"
assign	
number	42
end	
semicolon	

Beispiel: Zeichenmengen und Terminalklassen in Pascal

Zeichenmenge	Reguläre Grammatik	Regulärer Ausdruck
Zeichen	letter = "a" "b" ... "z"	[a-zA-Z]
Ziffern	digit = "0" "1" ... "9"	[0-9]
Ziffern (hex)	hex = "0" "1" ... "a" ... "F"	[0-9a-fA-F]

Terminalklasse	Reguläre Grammatik	Regulärer Ausdruck
Zahlen (int)	uint = digit { digit } "\$" hex { hex }	[0-9]+ \\$[0-9a-fA-F]+
Zahlen (real)	ureal = digit {digit} "." digit {digit} ["E" "e" ["+" "-"] digit {digit}]	[0-9]+\.[0-9]+([eE][-+]?[0-9]+)?
Namen	ident = letter { letter digit }	[a-zA-Z][a-zA-Z0-9]*
Zeichenketten	string = "" regStr ""	'[^']*'

4.2.2 Kontextfreie Grammatiken



Die Bezeichnung kontextfrei für eine Grammatik drückt aus, dass in allen Symbolketten, die aus dem Satzsymbol durch Anwendung der Ersetzungsregeln abgeleitet werden können, jedes darin vorkommende **Nonterminalsymbol unabhängig von den Symbolen links oder rechts** davon, also unabhängig von seiner Umgebung, dem Kontext, durch eine seiner Alternativen **ersetzt werden kann**

Alle Ersetzungsregeln haben die folgende Form

$$A = \dots$$

$$V_{NT} = \{A\}$$

Beispiel

$$\begin{aligned} S &= A \text{ ";"}. \\ A &= \text{"a"} B \mid B B \text{"b"}. \\ B &= \text{"b"} \mid \text{"a"} \text{"b"}. \end{aligned}$$

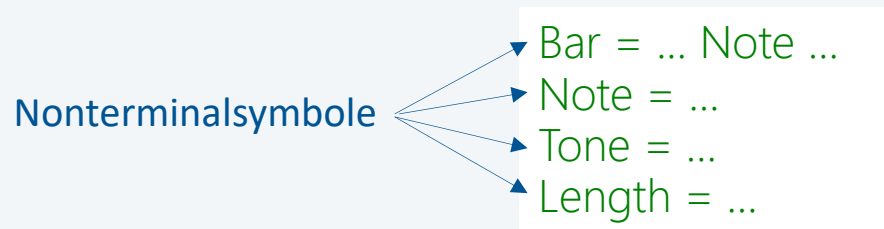
NT B kann unabhängig vom Kontext durch "b"
oder "a" "b" ersetzt

1. Terminalsymbole

- elementare Bestandteile der Sätze einer Sprache (lassen sich nicht weiter zerlegen)
- einfache Terminalsymbole (z.B. ",", ".", ":", "<=", "BEGIN")
- Terminalklassen (z.B. `ident`, `number`)

2. Nonterminalsymbole

- sind Bezeichner von Abstraktionen und werden zur Gliederung von (Satz-)Strukturen verwendet



3. Ableitungsregeln

- zu jedem Nonterminalsymbol gibt es eine Ableitungsregel, die beschreibt, in welche Teile die durch das Nonterminalsymbol bezeichnete Abstraktion zerlegt werden kann.
- Beispiel:

```
Bar = Note "," Note "," Note.  
Note = Tone Length.  
Tone = "c" | "d" | "e" | "f" | "g" | "a" | "h" .  
Length = "1" | "2" | "4" | "8".
```

4. Start- (Satz-)Symbol

- linke Seite der "ersten" Ableitungsregel ("oberstes Nonterminalsymbol", "höchste Abstraktion")
- Beispiel: Bar

EBNF = Erweiterte Backus-Naur-Form (Wirthsche EBNF)

Symbol	Bedeutung
=	trennt linke und rechte Seite einer Ableitungsregel
	trennt Alternativen $a \mid b \dots a \text{ oder } b$
.	schließt eine Ableitungsregel ab
(...)	klammert zusammengehörige Teile
[...]	Optionsklammern $[a] \dots a \mid \epsilon$ entweder a oder epsilon
{ ... }	Wiederholungsklammern $\{a\} \dots \epsilon \mid a \mid aa \mid aaa \mid \dots$

beliebig oft: Vorteil: man braucht nicht dauernd Rekursion

Beispiel 1: Grammatik der Binärzahlen und leere Kette (in EBNF)

$S = \{ B \}.$
 $B = "0" \mid "1".$

$S = \{ "0" \mid "1" \}.$

$S = B \mid \epsilon.$
 $B = "0" \mid "1" \mid "0" B \mid "1" B .$

Beispiel 2: Grammatik der Bezeichner

$S = \text{letter} \{ \text{letter} \mid \text{digit} \}.$

$S = \text{letter} \mid \text{letter } R.$
 $R = \text{letter} \mid \text{digit} \mid \text{letter } R \mid \text{digit } R.$

Beispiel: Grammatik einer einfachen Sprache (in EBNF)

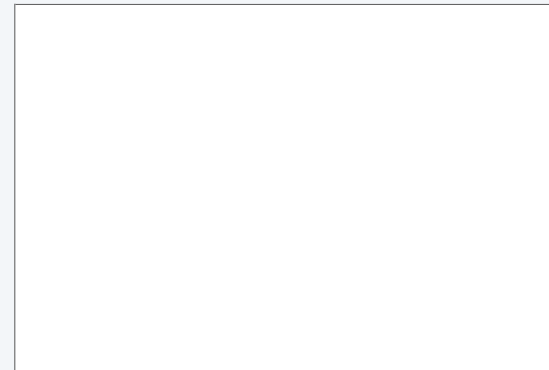


Menge aller Sätze, die mit "a" beginnen, mit "c" oder "d" enden und bei denen zwischen dem ersten und letzten Zeichen beliebig viele "b" vorkommen dürfen.

Grammatik:

```
S = "a" { "b" } ("c" | "d" )
```

Beispiele:



Rekursive Grammatik:

```
S = "a" B E.  
B = "b" B | ε .  
E = "c" | "d".
```



- Menge aller Sätze, die eine Folge von Zeiteinheiten definieren
- eine Zeiteinheit hat die Form H:M oder M, ein Komma trennt Zeiteinheiten
- H und M sind ein- oder zweistellige ganze Zahlen

Grammatik:

$V_T = \{ ":", ", ", \text{digit} \}$

$S = \text{Period} \{ ', ' \text{Period} \}$

$\text{Period} = \text{Number} \mid \text{Number} ":" \text{Number}.$

oder: $\text{Period} = [\text{Number} ":"] \text{Number}.$

$\text{Period} = \text{Number} [":" \text{Number}] //$ je nachdem sind die erste Number die Stunden oder Minuten

$\text{Number} = \text{digit} [\text{digit}]$

Beispiele:

45

2:30

45,2:30,3:15

34:45, 2:78,2:3

digit: [0-9]

-- kann nebenbei als Grammatik oder auch Ausdruck definiert werden, gehört nicht zur reg. Grammatik



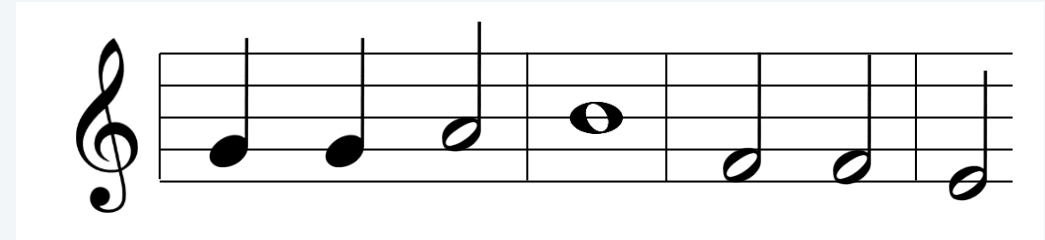
Beispiel: Musiknoten

- beliebige Anzahl von Takten
- beliebige Anzahl von Noten pro Takt
- Notennamen: c, d, e, f, g, a, h
- Notenwerte: 1, 2, 4, 8

```
|g4,g4,a2|h1|f2,f2|e2,f2|
```

```
|g4,g4,g8,g8,e8,e8|
```

```
|a2|
```



■ Grammatik $G(\text{Music})$

sStr := |

gute Lösung: gut abstrahiert, Trennsymbole auf Zeilenebene

Zeile = "|" Takt "{" Takt "|" }

Takt = Note {" , " Note }

Music = "|" Takt { Takt }

Takt = Note {" , " Note } "|" "

Definition: Wenn bei der Analyse in jeder Situation das aktuelle Terminalsymbol (genannt Vorgriffssymbol) ausreicht, um in der aktuellen Ersetzungsregel die richtige Alternative auszuwählen, erfüllt die Grammatik die so genannte **LL(1)-Bedingung**.

Beispiele

Eingabestrom

1 0 1



aktuelles
Terminalsymbol

3:15



aktuelles
Terminalsymbol

Ersetzungsregel

$P = \varepsilon \mid "0" \mid "1" \mid "0" P "0" \mid "1" P "1".$



zwei Alternativen starten mit dem TS "1"
LL(1)-Bedingung ist nicht erfüllt

$\text{Period} = \text{Number} \mid \text{Number} ":" \text{Number}.$



zwei Alternativen starten mit dem TS "digit"
LL(1)-Bedingung ist nicht erfüllt

LL(1)-Bedingung

- das bedeutet, dass in jeder Ersetzungsregel der Grammatik alle Alternativen mit unterschiedlichen Terminalsymbolen beginnen müssen
- auch bei Optionen und Wiederholungen muss mit einem Vorgriffssymbol die richtige Entscheidung getroffen werden können

Beispiel: Beseitigung von LL(1)-Konflikten durch "Zusammenfassen" von Alternativen

Period = Number | Number ":" Number.

LL(1)-Bedingung ist nicht erfüllt

Period = [Number ":"] Number.

LL(1)-Bedingung ist nicht erfüllt

Period = Number [":" Number].

LL(1)-Bedingung ist erfüllt

- eine kontextfreie Grammatik beschreibt lediglich die Syntax einer Sprache
- ein Syntaxanalysator kann daher nur prüfen, ob ein Eingabetext korrekt ist
- wenn man einen Eingabedatenstrom verarbeiten (das heißt, in eine andere Form transformieren und gegebenenfalls Ergebnisse aus ihm ermitteln) will, sind zusätzliche Aktionen erforderlich
- neben den Erkennungsprozessen (zur Analyse der syntaktischen Korrektheit des Eingabedatenstroms) benötigen wir Verarbeitungs-/Transformationsprozesse
- dazu erweitern wir kontextfreie Grammatiken zu **attributierten Grammatiken**
- eine attributierte Grammatik ist eine (kontextfreie) Grammatik, die mit **semantischen Aktionen** und **Attributen** so angereichert ist, dass damit der Verarbeitungs-/Transformationsprozess vollständig beschrieben ist

Semantische Aktionen beschreiben Operationen, die während der Erkennung des Eingabedatenstroms ausgeführt werden sollen

- Semantische Aktionen werden direkt an den **passenden Stellen** in die kontextfreie Grammatik eingefügt
- eine semantische Aktion unmittelbar nach einem Grammatiksymbol bedeutet, dass sie **nach der Erkennung dieses Symbols** (im Zuge der Syntaxanalyse) **ausgeführt** werden soll
- Schreibweise: Pseudocode zwischen **sem** und **endsem**

Beispiel: Differenz zwischen der Anzahl von 0en und 1en einer Binärzahl

```
S = { "0" | "1" }.
```

```
S =      sem int d = 0; endsem
  { "0"   sem d = d - 1; endsem
    | "1"   sem d = d + 1; endsem
  }      sem IO.print(d); endsem
  .
```

Beispiel: Differenz zwischen der Anzahl von 0en und 1en einer Binärzahl

```
S =      sem int d = 0; endsem
      { "0" sem d = d - 1; endsem
      | "1" sem d = d + 1; endsem
      }
      sem IO.print(d); endsem
      .
```

Eingabestrom	d	Ausgabe
11100	0	
11100	1	
11100	2	
11100	3	
11100	2	
11100	1	1

Mit semantischen Aktionen können also Ausführungen beliebiger Aktionen von der Struktur des Eingabedatenstroms abhängig gemacht werden

Attribute beschreiben Eigenschaften von Grammatiksymbolen in Form von **Datenobjekten**, die den **Grammatiksymbolen** zugeordnet werden

Bei den Attributen unterscheiden wir nach ihrer Art

- **semantische Attribute** sind mit Nonterminalsymbolen der Grammatik verknüpft, deren Werte werden mit semantischen Aktionen im Zuge des Erkennungsprozesses ermittelt

Beispiele: `Mail`_{↑nrWords ↑nrLongWords} `Period`_{↑value}

- **lexikalische Attribute** sind mit Terminalklassen verknüpft, deren Werte werden vom lexikalischen Analysator ermittelt und zur Verfügung gestellt

- Beispiele: `ident`_{↑name} `number`_{↑value} `digit`_{↑value}

Ausgangsattribute: Wenn ein `ident` erkannt wird, steht z.B. ein `name` zur Verfügung

Nonterminalsymbole können auch Eingangsattribute haben

- Ausgangsattribute ($S_{\uparrow x}$) ... entstehen bei der Erkennung eines Terminal- oder Nonterminalsymbols

```
N↑n = ... B↑c B↑d sem n = 2*c + d; endsem.
```

```
B↑v = ... | "1" sem v = 1; endsem.
```

```
... = digit↑value sem IO.print(value); endsem.
```

... Terminalklasse $\text{digit}_{\uparrow \text{value}}$
-- Klasse von symbole

Datenflüsse zwischen Nonterminalsymbole

- Eingangsattribute ($S_{\downarrow y}$) ... werden einem Nonterminalsymbol zu seiner Erkennung mitgegeben

```
... = ... sem n = 42; endsem F↓n ...
```

```
F↓n = ... sem ... = n; endsem.
```

Beispiel: Länge eines Palindroms

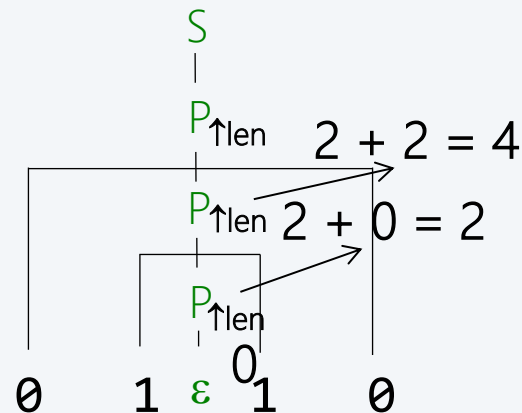
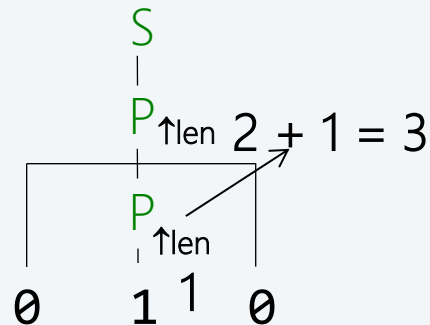


Grammatik $G(S)$ $S = P. \quad P = \varepsilon \mid "0" \mid "1" \mid "0" P "0" \mid "1" P "1".$

Attributierte Grammatik

```
S = P_len      sem IO.print("Länge = " + Len); endsem
P_len = ep
```

"Dekorierter" Syntaxbaum



Beispiel: Berechnung des Werts einer Hex-Zahl



Grammatik (z.B. 1Bh)

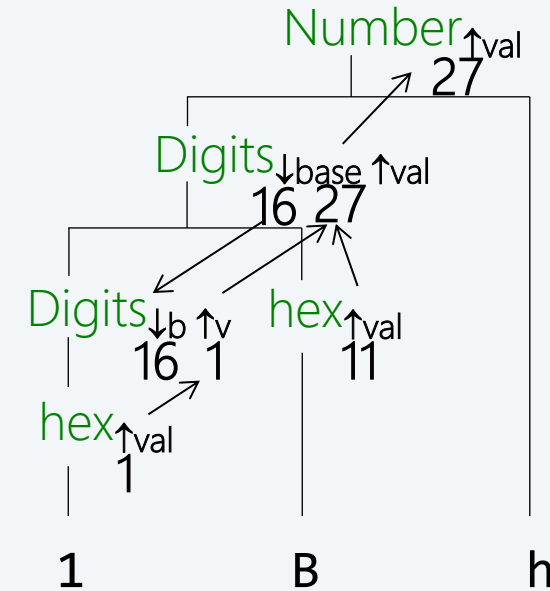
Number = Digits | Digits "h".
Digits = hex | Digits hex.

Terminalklasse hex ("0" | "1" | ... | "F")

Attribute

Number^{↑val} Digits^{↓base ↑val} hex^{↑val}

Attributierte Grammatik



Beispiel: Grammatik für Binärzahlen und leere Kette

$S = \{ B \}.$
 $B = "0" \mid "1" .$

Attributierte Grammatik

$S_{\uparrow \text{value}} =$ sem value = 0; endsem
 $\{ B_{\uparrow b}$ sem value = 2*value + b; endsem
 $\}.$
 $B_{\uparrow b} =$ "0" sem b = 0; endsem
 | "1" sem b = 1; endsem
 .

Beispiel-Berechnung

	Eingabestrom	b	value
1010		?	0
▲			
1010		1	1
▲			
1010		0	2
▲			
1010		1	5
▲			
1010		0	10
▲			

Grammatik G(S):

```
S = Period { "," Period }.  
Period = Number [ ":" Number ].  
Number = digit [ digit ].
```

Terminalsymbole:

"," , ":" , digit ("0" | "1" | "2" | ... | "9")

Attribute: $\text{Period}_{\uparrow p}$ $\text{Number}_{\uparrow n}$ $\text{digit}_{\uparrow d}$

Attributierte Grammatik:

```
S = Period↑p1  
  { "," Period↑p2  
  }
```

```
Period↑p = Number↑n  
  [ ":" Number↑m  
  ].
```

```
Number↑n = digit↑d  
  [ digit↑d  
  ].
```

```
sem int sum = p1; endsem  
sem sum = sum + p2; endsem  
sem writeTime(sum); endsem
```

```
sem p = n; endsem  
sem p = 60 * n + m; endsem
```

```
sem n = d; endsem  
sem n = 10 * n + d; endsem
```

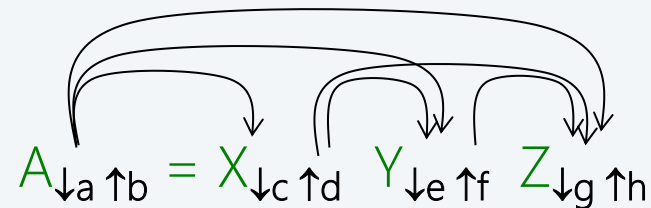
56
2:15
 $\uparrow_n$ $\uparrow_m$

Eine attributierte Grammatik ist **L-attributiert**, wenn für jede ihrer Regeln

$$A = X_1 \dots X_n$$

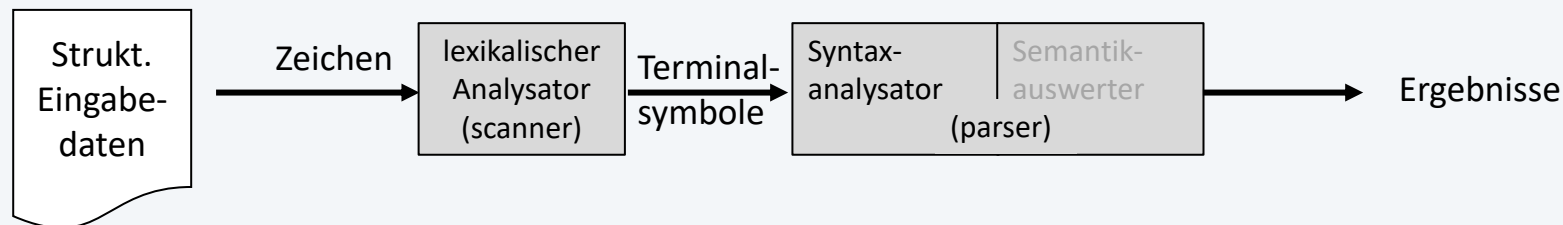
gilt: die Eingangsattribute von X_i , für alle $1 \leq i \leq n$ hängen nur von den Eingangsattributen von A und den Ausgangsattributen von $X_1 \dots X_{i-1}$ ab

Beispiel:



L-attributierte Grammatiken ermöglichen eine mit der Syntaxanalyse von links nach rechts schritthaltende Attributauswertung ohne expliziten, dekorierten Syntaxbaum

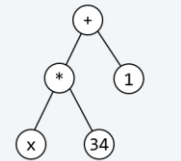
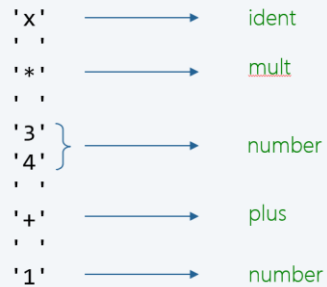
- Ableitungsregeln werden als Erkennungsprozesse aufgefasst
- durch die Grammatik sind alle (im Zuge der Verarbeitung eines Eingabedatenstroms) notwendigen Erkennungsprozesse festgelegt
- die implementierten Erkennungsprozesse zusammengekommen nennt man Syntaxanalysator
- der Syntaxanalysator setzt eine Programmkomponente (scanner) voraus, die Terminalsymbole erkennt und liefert



Arbeitsweise

- jede Ableitungsregel wird von links nach rechts durchlaufen
- zur Alternativenauswahl wird das aktuelle Eingabesymbol verwendet
- nach Erkennung eines Terminalsymbols fordert der Syntaxanalysator vom lexikalischen Analysator das nächste Element (Terminalsymbol) des Eingabedatenstroms an
- bei jedem Nonterminalsymbol
 - unterbricht der Syntaxanalysator die Abarbeitung der laufenden Regel
 - arbeitet das angetroffene Nonterminalsymbol ab, d.h. führt den entsprechenden Erkennungsprozess aus
 - setzt dann an der Unterbrechungsstelle fort

4 Datenorientierter Systementwurf



Annahme: $x = 2$

4.1 Denkmodell

4.2 Formale Sprachen und Grammatiken

4.3 Systementwurf mit ATGen

4.4 Transformation ATGen in Java

Vorgehensweise

Schritt 1: Syntaktische Struktur der Eingabe als kontextfreie Grammatik darstellen

Schritt 2: Attribute für den Verarbeitungsprozess finden

Schritt 3: Semantische Aktionen entwerfen und in die Grammatik integrieren

Schritt 4: Code für Systemimplementierung ermitteln/generieren

Aufgabenstellung für Entwurfsbeispiel

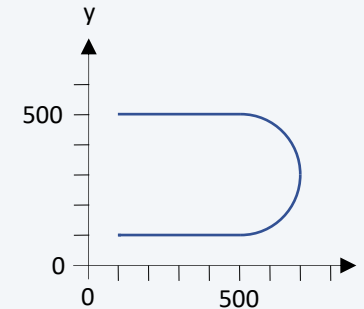
Zu Simulations- und Bedienungs Zwecken soll eine Anwendung für einen 3D-Drucker entwickelt werden.



- der Bewegungsablauf des 3D-Druckers (mit 3 Achsen) wird durch eine spezielle (vorgegebene) Sprache beschrieben
- Beschreibung der Bewegungsbahn: Die Bewegungsbahn ist eine Folge von linearen Teilschritten; zwei Bahnformen können beschrieben werden:
 - lineare Bewegung zum Punkt (x, y, z) in n Schritten: $n \text{ L } (x, y, z)$
Beispiel: $5 \text{ L } (100, 200, 300)$
 - Kreisbewegung zum Punkt (x, y, z) über einen Zwischenpunkt (x_1, y_1, z_1) in n Schritten: $n \text{ C } (x_1, y_1, z_1) (x, y, z)$
Beispiel: $20 \text{ C } (50, 50, 50) (100, 100, 100)$
- Koordinaten werden in 1/10 mm angegeben (z.B. 123 = 12.3 mm)
- der Endpunkt einer Bahn ist gleichzeitig der Anfangspunkt der nächsten Bahn

Beispiel 1

```
(100,100,100)
10 L (500,100,100)
20 C (700,300,100)(500,500,100)
10 L (100,500,100)
E
```



Beispiel 2

```
(200,100,100)
5 L (-100,+200,-100)
20 C (+700,+700,+1200) (+0,+1400,+0)
E
```

Start beim Punkt $x=200, y=100, z=100$

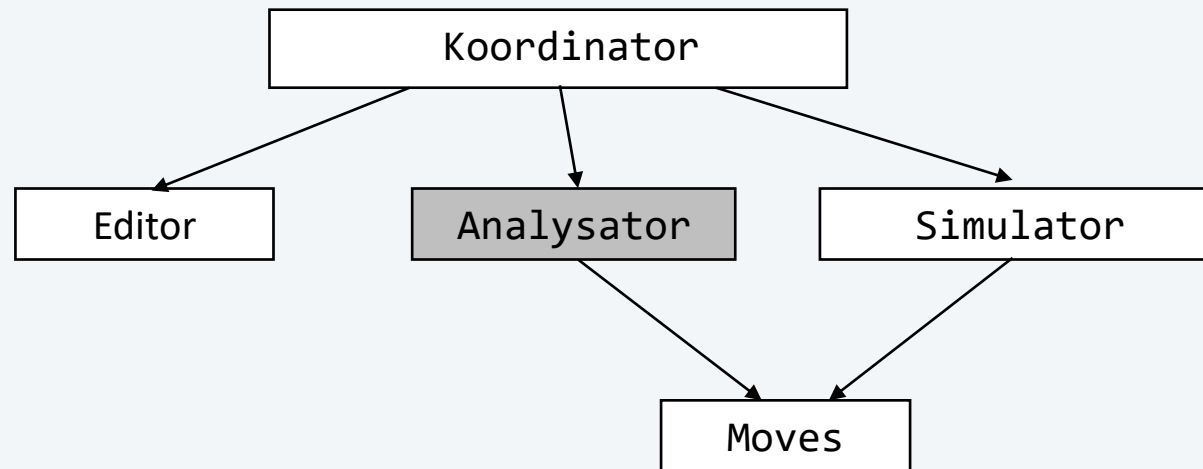
Lineare Bewegung in 5 Schritten zum
Punkt $x=100, y=300, z=0$

Kreisbewegung in 20 Schritten zum
Punkt $x=100, y=1700, z=0$

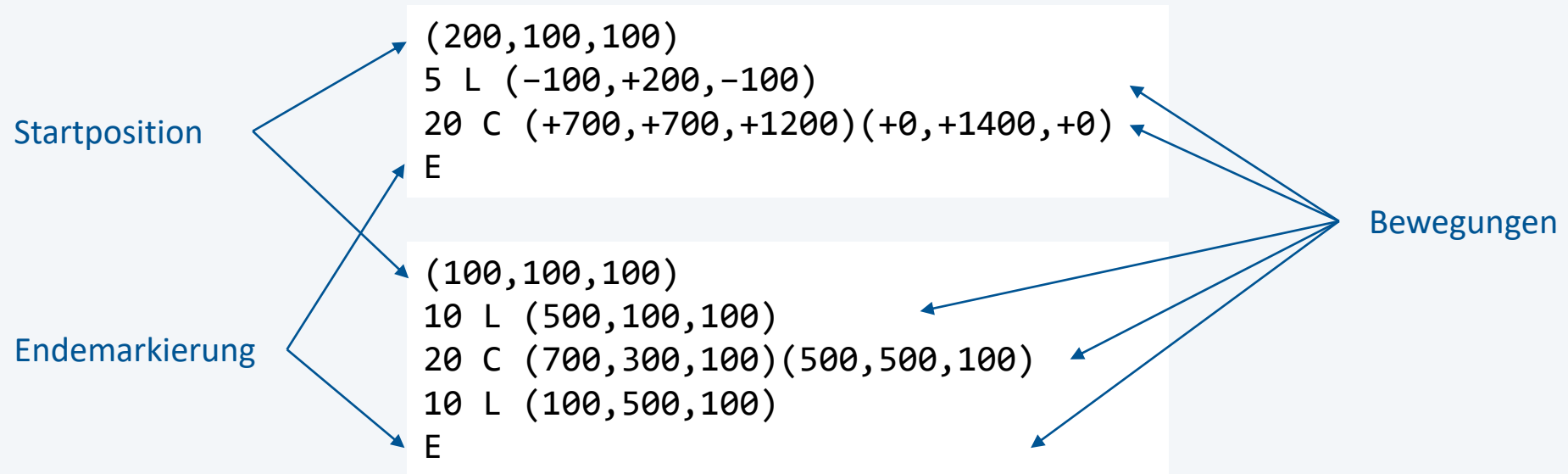
der Punkt $x=800, y=1000, z=1200$
soll auf der Kreisbahn liegen

Entwurf der Analysator-Komponente der 3D-Drucker-Anwendung

Die Analysator-Komponente dient dazu, eine gegebene Bewegungsbeschreibung auf syntaktische Korrektheit zu prüfen und die durch sie spezifizierten Bewegungssegmente in geeigneter Form in der Datenkapsel Moves abzuspeichern (damit die Simulatorkomponente auf eine syntaktisch korrekte Bewegungsbeschreibung zugreifen kann).



Gesamte Bewegungsbeschreibung (Startsymbol)



PrinterMoves = Position { Move } "E".

Positionsangabe

Beispiel: (200,100,100)

(200,100,100)

(200,100,100)

Position = "(" ... "," ... "," ... ")".

Coordinate

Position = "(" Coordinate "," Coordinate "," Coordinate ")".

Teilbewegung

10 L (-100,+200,-100)

Move = Number "L" Position.

20 C (700,300,100)(500,500,100)

Move = Number "C" Position Position.

Move = Number ("L" Position | "C" Position Position).

Koordinate 200 -100 +200

Coordinate = ["+" | "-"] Number.

Zahl 200 100

Number = digit { digit } .

Grammatik $G(\text{PrinterMoves})$:

$\text{PrinterMoves} = \text{Position} \{ \text{Move} \} "E"$.

$\text{Position} = "(" \text{Coordinate} "," \text{Coordinate} "," \text{Coordinate} ")"$.

$\text{Move} = \text{Number} ("L" \text{Position} \mid "C" \text{Position} \text{Position})$.

$\text{Coordinate} = ["+" \mid "-"] \text{Number}$.

$\text{Number} = \text{digit} \{ \text{digit} \}$.

Alphabet (Terminalsymbole):

- $"L", "C", "E", "+, -, (,), ,, \text{digit}$
- Terminalklasse $\text{digit} = "0" \mid "1" \mid \dots \mid "9"$

Nonterminalsymbole (Startsymbol PrinterMoves):

- $\text{PrinterMoves}, \text{Position}, \text{Move}, \text{Coordinate}, \text{Number}$

PrinterMoves

erkennt eine vollständige Bewegungsbeschreibung

Position $\downarrow x_0 \downarrow y_0 \downarrow z_0 \uparrow x \uparrow y \uparrow z$

erkennt eine Position mit den Koordinaten x , y und z . x_0 , y_0 und z_0 bedeuten die Ursprungsposition, auf die sich relative Koordinatenangaben beziehen

Move $\downarrow x_0 \downarrow y_0 \downarrow z_0 \uparrow x \uparrow y \uparrow z$

erkennt eine Bewegung beginnend bei (x_0, y_0, z_0) und liefert den Endpunkt (x, y, z) der Bewegung

Coordinate $\downarrow v_0 \uparrow v$

erkennt eine Koordinate mit dem Wert v . v_0 ist ein Ursprungswert, auf den sich relative Angaben beziehen

Number $\uparrow v$

erkennt eine ganze Dezimalzahl mit dem Wert v

digit $\uparrow d$

erkennt eine Ziffer mit dem Wert d

Semantische Aktionen entwerfen und in Grammatik integrieren

```
PrinterMoves =  
    Position↓0 ↓0 ↓0 ↑x0 ↑y0 ↑z0 sem Moves.setStartPosition(x0, y0, z0); endsem  
    { Move↓x0 ↓y0 ↓z0 ↑x ↑y ↑z sem x0 = x; y0 = y; z0 = z; endsem  
    } "E".
```

```
public class Moves {  
    public static void setStartPosition(int x, int y, int z) {...}  
}
```

Semantische Aktionen entwerfen und in Grammatik integrieren

Position_{↓x0 ↓y0 ↓z0 ↑x ↑y ↑z} =
"(" sem v0 = x0; endsem Coordinate_{↓v0 ↑v} sem x = v; endsem
"," sem v0 = y0; endsem Coordinate_{↓v0 ↑v} sem y = v; endsem
"," sem v0 := z0; endsem Coordinate_{↓v0 ↑v} sem z = v; endsem
")."

Einfacher

Position_{↓x0 ↓y0 ↓z0 ↑x ↑y ↑z} =
"(" Coordinate_{↓x0 ↑x}
"," Coordinate_{↓y0 ↑y}
"," Coordinate_{↓z0 ↑z}
")."

Semantische Aktionen entwerfen und in Grammatik integrieren

```
Move↓x0 ↓y0 ↓z0 ↑x ↑y ↑z =  
  Number↑steps  
  ( "L" Position↓x0 ↓y0 ↓z0 ↑x ↑y ↑z      sem Moves.addLine(x, y, z, steps); endsem  
  
  | "C" Position↓x0 ↓y0 ↓z0 ↑x1 ↑y1 ↑z1  
    Position↓x0 ↓y0 ↓z0 ↑x ↑y ↑z      sem Moves.addArc(x, y, z, x1, y1, z1, steps) endsem  
  ).
```

```
public class Moves {  
    public static void setStartPosition(int x, int y, int z) {...}  
    public static void addLine(int x, int y, int z, int steps) {...}  
    public static void addArc(int x, int y, int z,  
                             int x1, int y1, int z1, int steps) {...}  
}
```

Semantische Aktionen entwerfen und in Grammatik integrieren

```
Coordinate↓v0 ↑v =  
    [ "+"      sem char sign = ' '; endsem  
      | "-"      sem sign = '+'; endsem  
      | "-"      sem sign = '-'; endsem  
    ]  
Number↑v sem if (sign == '+')  
            v = v0 + v;  
            else if (sign == '-')  
                v := v0 - v;  
            endsem
```

.

Semantische Aktionen entwerfen und in Grammatik integrieren

```
Number↑v =  
    digit↑d    sem v = d; endsem  
    { digit↑d    sem v = 10 * v + d; endsem  
    }.
```


Abstrakte semantische Aktionen spezifizieren

Klasse Moves zur Verwaltung einer Liste von Teilbewegungen mit folgenden Klassenmethoden, die als semantische Aktionen im Systementwurf vorkommen:

```
void setStartPosition(int x, int y, int z)
```

Initialisiert die Teilbewegungsliste mit dem Startpunkt (x, y, z).

```
void addLine(int x, int y, int z, int steps)
```

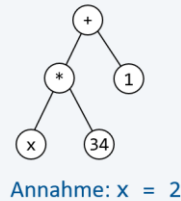
Erweitert die Teilbewegungsliste um eine lineare Bewegung zum Punkt (x, y, z) in steps Schritten.

```
void addArc(int x, int y, int z, int x1, int y1, int z1, int steps)
```

Erweitert die Teilbewegungsliste um eine kreisförmige Bewegung über den Punkt (x1, y1, z1) zum Punkt (x, y, z) in steps Schritten.

4 Datenorientierter Systementwurf

'x'	→	ident
'*'	→	mult
'3'	}	number
'4'		
'+'	→	plus
'1'	→	number



4.1 Denkmodell

4.2 Formale Sprachen und Grammatiken

4.3 Systementwurf mit ATGen

4.4 Transformation ATGen in Java

- Aufzählungstyp Symbol für Terminalsymbole (inkl. Terminalklassen)

```
public enum Symbol {  
    ERROR, EOF,  
    DIGIT, E, L, C, LEFT_PARENTHESIS, RIGHT_PARENTHESIS, COMMA, PLUS, MINUS  
}
```

- Klasse Scanner (lexikalischer Analysator)

```
public class Scanner {  
    public Symbol sy;           // current terminal symbol  
    public int digitValue;     // attribute value if sy == digitSy  
  
    public Scanner(...) {...}  // initialize sy with first terminal symbol  
    public void nextSy() {...} // provide code sy for next terminal symbol  
}
```

Klasse Parser (Syntaxanalysator)

```
public class Parser {  
    private Scanner scanner;  
  
    public Parser(Scanner scanner) {  
        this.scanner = scanner;  
    }  
  
    public void PrinterMoves() {...}  
    private void Position() {...}  
    private void Move() {...}  
    private void Coordinate() {...}  
    private void Number() {...}  
  
    private void expect(Symbol expectedSy) {  
        if (scanner.sy != expectedSy) {  
            throw new SyntaxError();  
        }  
    }  
}
```

eine Parsermethode für jede Grammatikregel

Nonterminalsymbol <code>A = ... Move ...</code>	Aufruf der entsprechenden Parsermethode <pre>private void A() { Move(); }</pre>
Terminalsymbol <code>A = ... 'L' ...</code>	Symbol prüfen und nächstes Symbol anfordern <pre>private void A() { expect(Symbol.L); scanner.nextSy(); }</pre>
Alternativen <code>A = ... ('+' '-') ...</code>	switch-Anweisung (oder Verzweigungen) <pre>private void A() { switch (scanner.sy) { case Symbol.PLUS: scanner.nextSy(); ... break; case Symbol.MINUS: scanner.nextSy(); ... break; default: throw new SyntaxError(); } }</pre>

Option <code>A = ... ['-'] ...</code>	Verzweigung <pre>private void A() { if (scanner.sy == Symbol.MINUS) { scanner.nextSy(); ... } }</pre>
Wiederholung <code>A = ... { digit } ...</code>	Schleife <pre>private void A() { while (scanner.sy == Symbol.DIGIT) { scanner.nextSy(); } }</pre>
<code>A = ... { Move } ...</code>	<pre>private void A() { while (scanner.sy == Symbol.DIGIT) { Move(); } }</pre>

`Move = Number ...`

`Number = digit { digit } .`

Eingangsattribute $A_{\downarrow x \downarrow y \downarrow z} = \dots B_{\downarrow 2 * x} \dots$	Eingangsparameter <pre>private void A(int x, int y, int z) { B(2*x); }</pre>
Ausgangsattribut $A_{\uparrow x} = \dots B_{\uparrow y} \dots$	Rückgabewert <pre>private int A() { int y = B(); int x = ...; return x; }</pre>
Ausgangsattribut $A = \dots \text{digit}_{\uparrow d} \dots$	Rückgabewert <pre>private void A() { expect(Symbol.DIGIT); int d = scanner digitValue; scanner.nextSy(); }</pre>
Semantische Aktion $A = \dots \text{sem } x = y; \text{ endsem}$	Codestücke oder Algorithmen <pre>private void A() { x = y; }</pre>

```
PrinterMoves =  
  Position  
  { Move  
  }  
  "E" .
```

Move = Number ...

Number = digit { digit } .

```
public void PrinterMoves() {  
  Position();  
  while (scanner.sy == Symbol.DIGIT) {  
    Move();  
  }  
  expect(Symbol.E);  
  scanner.nextSy();  
  expect(Symbol.EOF);  
}
```



```
PrinterMoves =  
  Position ↓0 ↓0 ↓0 ↑x0 ↑y0 ↑z0  
  sem  
    Moves.setStart...  
  endsem  
{ Move ↓x0 ↓y0 ↓z0 ↑x ↑y ↑z  
  sem  
    x0 = x; y0 = y; z0 = z;  
  endsem  
}  
"E" .
```

```
public void PrinterMoves() {  
  Position p = null;  
  Position p0 = Position(new Position(0, 0, 0));  
  Moves.setStartPosition(p0.x, p0.y, p0.z);  
  while (scanner.sy == Symbol.DIGIT) {  
    p = Move(p0);  
    p0 = p;  
  }  
  expect(Symbol.E);  
  scanner.nextSy();  
  expect(Symbol.EOF);  
}
```

Von der Grammatikregel zur Erkennungsprozedur (manuell)

```
Coordinate↓v0 ↑v =  
    sem char sign = ' '; endsem  
    [ "+" sem sign = '+'; endsem  
    | "-" sem sign = '-'; endsem  
    ]  
Number↑v  
    sem  
        if (sign == '+')  
            v = v0 + v;  
        else if (sign == '-')  
            v = v0 - v;  
    endsem  
.
```

```
private int Coordinate(int value0) {  
    char sign = ' ';  
    if (scanner.sy == Symbol.PLUS) {  
        sign = '+';  
        scanner.nextSy();  
    } else if (scanner.sy == Symbol.MINUS) {  
        sign = '-';  
        scanner.nextSy();  
    }  
    int value = Number();  
    if (sign == '+') {  
        value = value0 + value;  
    }  
    else if (sign == '-') {  
        value = value0 - value;  
    }  
    return value;  
}
```

```
public static boolean analyzeText(String text) {  
    BufferedReader reader = new BufferedReader(new StringReader(text));  
    Scanner scanner = new Scanner(reader);  
    Parser p = new Parser(scanner);  
    try {  
        p.printerMoves();  
        return true;  
    } catch (SyntaxError e) {  
        IO.println("invalid input");  
        return false;  
    }  
}
```

Von der attributierten Grammatik zur Systemimplementierung

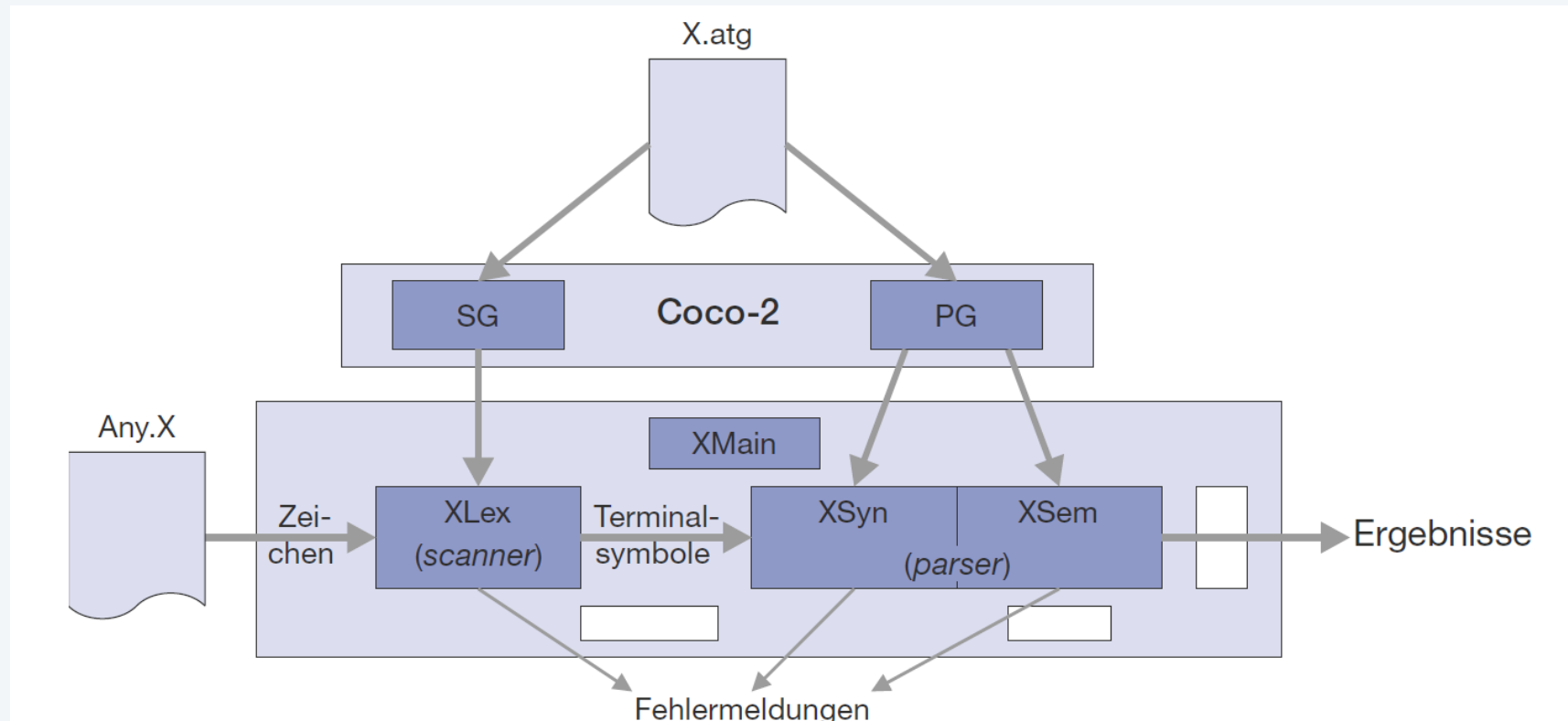
Schon in den 1970-er Jahren wurden Softwarewerkzeuge entwickelt, die den Compilerbauer bei der Herstellung wesentlicher Teile eines Compilers unterstützen, indem sie aus formalen Beschreibungen Quelltext für die dadurch spezifizierten Teile erzeugen.

Bekannte und verbreitete Compilerbau-Werkzeuge, auch Compiler-Generatoren oder Compiler-Compiler genannt, sind:

- Lex zur Generierung lexikalischer Analysatoren aus der Beschreibung lexikalischer Strukturen durch reguläre Ausdrücke
- Yacc zur Generierung von Bottom-up-Syntaxanalysatoren aus attributierten Grammatiken (kontextfreie LALR(1)-Grammatiken)
- Der Scanner-Generator Alex und der Parser-Generator Coco zur Herstellung von Top-down-Syntaxanalysatoren aus attributierten Grammatiken auf Basis kontextfreier LL(1)-Grammatiken
- Coco/R, der als Syntaxanalyseverfahren den rekursiven Abstieg nutzt und daher sehr effizient ist
- ANTLR zur Herstellung von Syntaxanalyseverfahren und von Transformationsprozessen

Von der attributierten Grammatik zur Systemimplementierung

Beispiel: Coco-2 für unterschiedliche Programmiersprachen (z. B. Pascal, C/C++ und C#)



7.5 Vorgehensmodell zur datenorientierten Programmierung

1. Beschreiben der **lexikalischen Struktur** des Eingabedatenstroms (vorzugsweise mit **regulären Grammatiken** oder **regulären Ausdrücken**)
2. Generierung eines **lexikalischen Analysators** und Bereitstellung eines rudimentären Koordinationsrahmens, damit der lexikalische Analysator getestet werden kann
3. Beschreiben der **syntaktischen Struktur** des Eingabedatenstroms mit einer **kontextfreien Grammatik**, die die **LL(1)-Bedingung** erfüllt
4. Generierung eines **Syntaxanalysators** und Erweiterung des Koordinationsrahmens, damit der Syntaxanalysator getestet werden kann

5. Festlegen der **lexikalischen** und der **semantischen Attribute** und Erweiterung der kontextfreien Grammatik mit den für den beabsichtigten Transformationsprozess notwendigen **semantischen Aktionen** (unter Beachtung, dass die Bedingung für eine L-attributierte Grammatik erfüllt wird); das Ergebnis ist eine **attributionierte Grammatik**.
Hinweis: Unter Umständen müssen auch semantische Algorithmen entworfen werden, wenn man mit einfachen semantischen Aktionen nicht das Auslangen findet (es ist sinnvoll, diese in einem eigenen Modul zusammenzufassen)
6. Erweiterung des lexikalischen Analysators, so dass auch die **Werte der lexikalischen Attribute** ermittelt werden können und Generierung der vollständigen Parserkomponente (d.h. inklusive Semantikauswerter).
Erweiterung des Koordinationsrahmens, so dass damit das gesamte Programmsystem getestet werden kann

Attributierte Grammatiken sind (ein aus dem Compilerbau bekanntes Konzept) geeignet für:

- die Spezifikation von Softwaresystemen
- den Entwurf von Softwaresystemen
- die Dokumentation von Softwaresystemen

Attributierte Grammatiken bzw. datenorientierte Programmierung sind dann nutzbringend einsetzbar:

- wenn im Wesentlichen ein Eingabedatenstrom vorliegt
- wenn der Eingabedatenstrom genügend strukturiert ist
- wenn im Wesentlichen der Eingabedatenstrom in eine andere Form transformiert werden soll

Vorteile

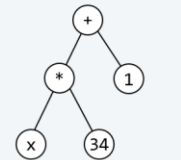
- Deskriptive Entwurfstechnik
- Modularisierungstechnik
- zwingt zur Trennung von Syntax und Semantik, Syntax ist Routine, die/der Entwickler*in kann sich auf Semantik konzentrieren
- knappe und (mit etwas Übung) gut lesbare Darstellung (hervorragendes Dokumentationsmittel)
- Systementwurf lässt sich automatisch in Programme transformieren

Nachteile

- nicht anwendbar bei mehreren parallel zu verarbeitenden Eingabedatenströmen
- Anwendung setzt Grundkenntnisse des Übersetzerbaus voraus

4 Datenorientierter Systementwurf

'x'	→	ident
'*'	→	mult
'3'	}	number
'4'		
'+'	→	plus
'1'	→	number



Annahme: $x = 2$

- 4.1 Denkmodell
- 4.2 Formale Sprachen und Grammatiken
- 4.3 Systementwurf mit ATGen
- 4.4 Transformation ATGen in Java